ADS 4 - Truck Platooning



sea:me

software engineering in automotive
and mobility ecosystems



0. Introduction



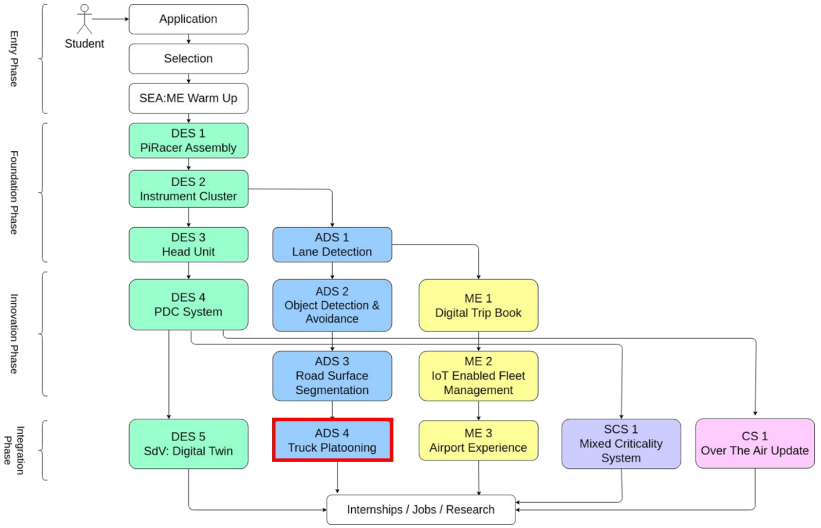
Tony (Cho SeokHun)

Focus Area: Perception
- Lane Detection, Dynamic ROI, Scenario



Justin (Jung JunBeom)

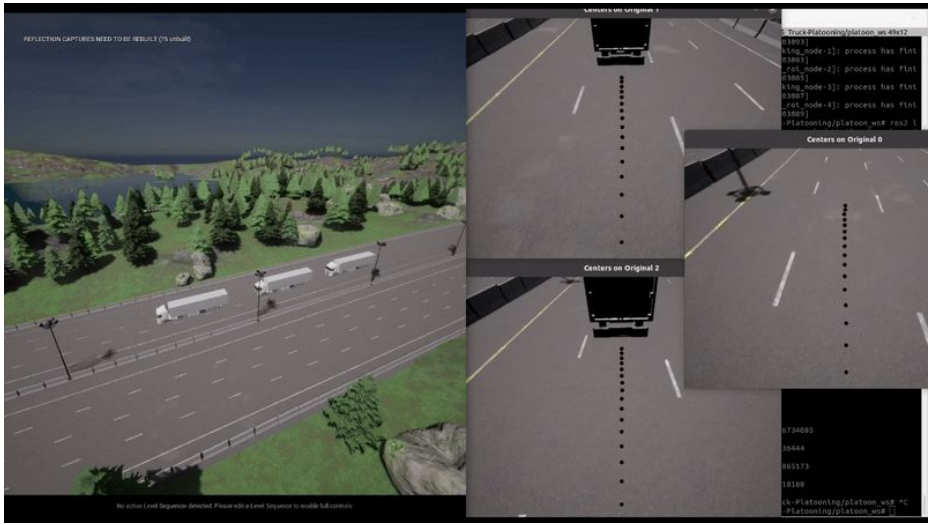
Focus Area: Control
- Gap, Velocity Control, Lidar, Scenario



What is Truck Platooning?



Why Truck Platooning Matters



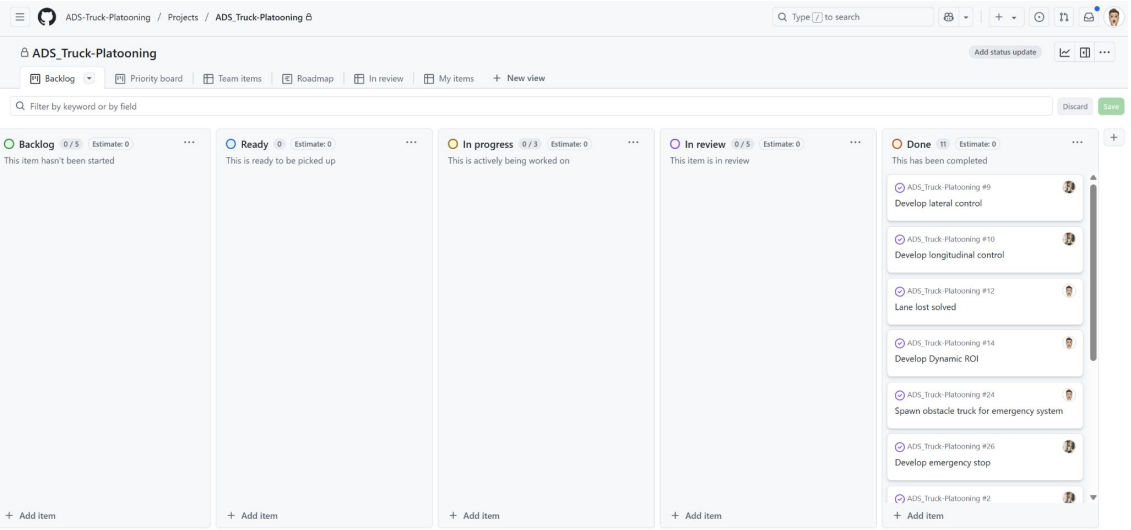
1. Fuel Savings & Lower Carbon Emissions
2. Improved Road Efficiency
3. Enhanced Safety

1. Platooning reduces aerodynamic drag, cutting fuel use by 10–15% for trailing vehicles.
2. Vehicles travel closer together, increasing road capacity and reducing traffic congestion.
3. Automated systems maintain consistent speeds and safe distances between vehicles.

Contents

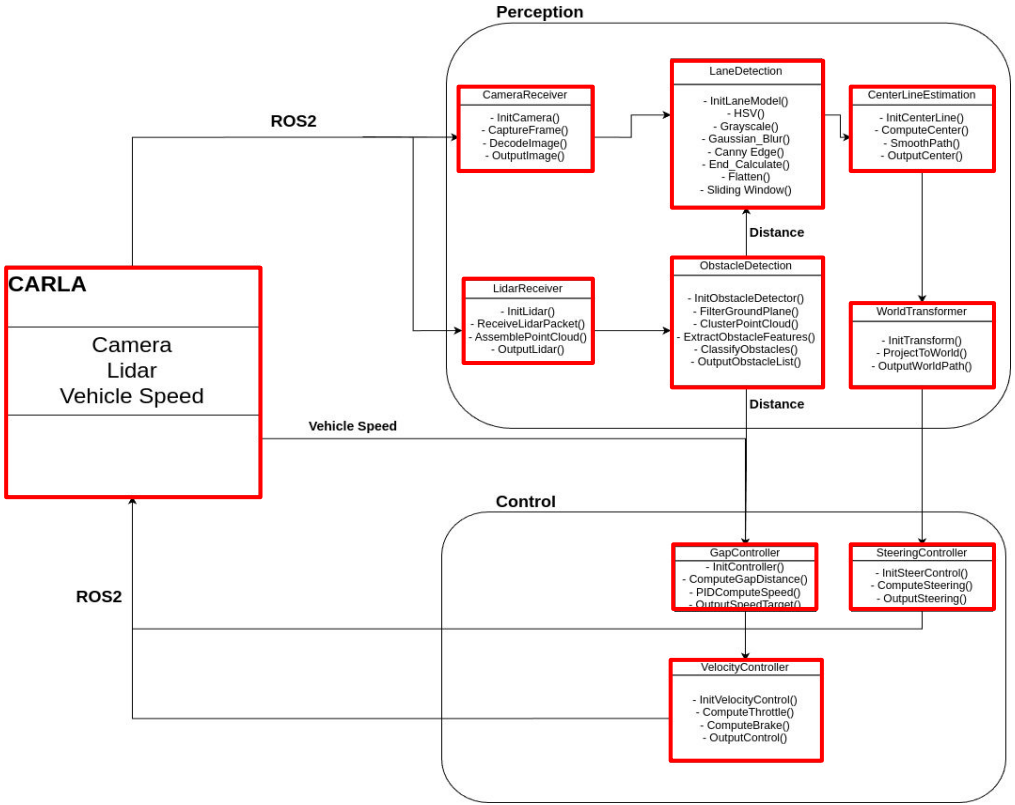
1. How we work
2. System Architecture
3. Timeline & Tech Stack
4. Perception
5. Control
6. Scenario
7. Q&A

1. How we work



We mainly use Github and Notion for collaboration

2. System Architecture



3. Timeline

Week 1

- CARLA setup & installation
- Dockerfile creation
- **Develop basic lane detection**

Week 2

- Lane detection improvements

Week 3

- Convert lane center pixel to world coordinates
- **Develop lateral control**

Week 4

- Develop LiDAR obstacle detector
- **Develop longitudinal control**
- **Develop basic Dynamic ROI**

Week 5

- **Develop Dynamic ROI**
- **Develop longitudinal control**
- Tune all modules and fix bugs
- **Develop two driving scenarios**

Week 6

- **Develop two driving scenarios**
- Conduct integration testing

3. Tech Stack

Simulation



CARLA

Sensors



Camera, LiDAR

Development



Python, C++, OpneCV, Obstacle detector package(for LiDAR Detection), Pure Pursuit(for Control)

Middleware



ROS2

Communication



ROS2-based V2V

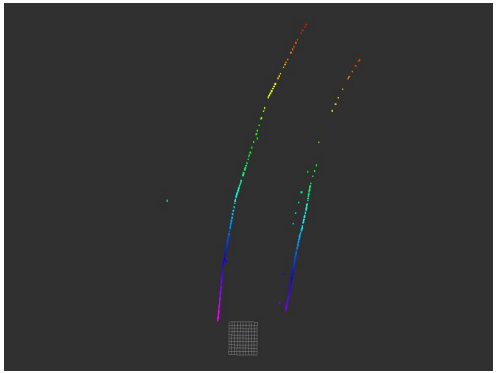
4. Perception



Sensors



Camera, LiDAR



Detect ego lanes
for lateral control

Measure
longitudinal
distance of the
preceding trailer

4. Perception

Lane Detection Objective

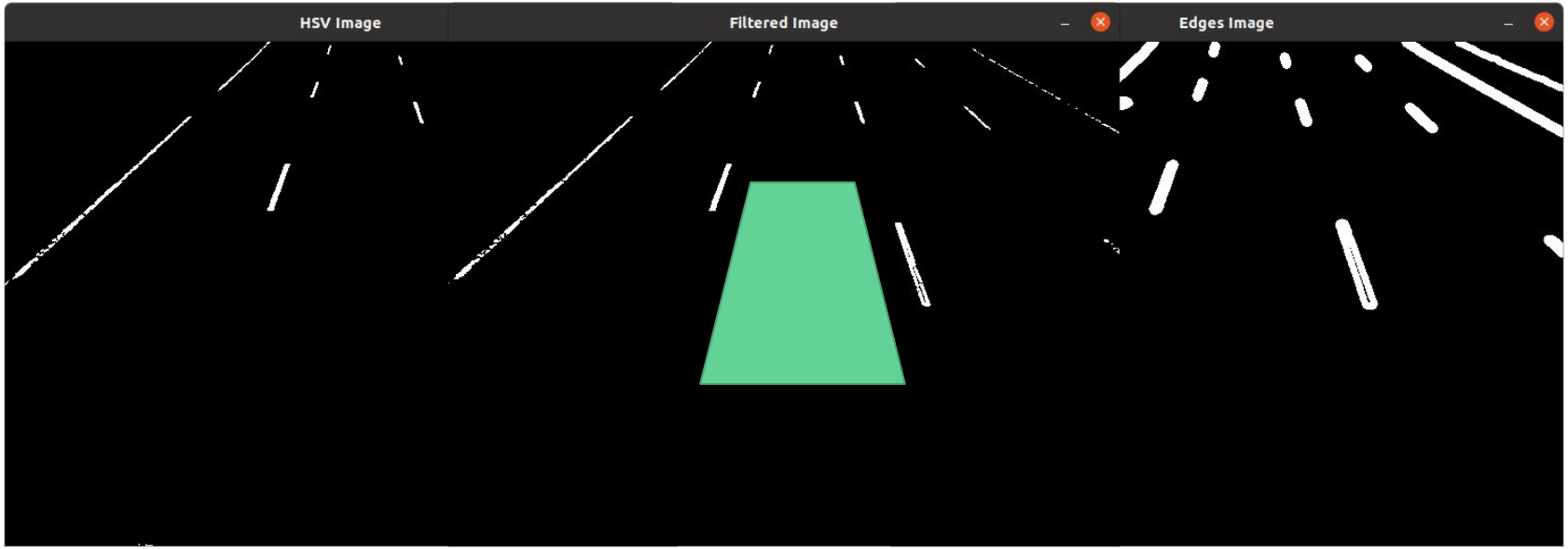
Get Camera input → Estimate lane curvature → Feed into vehicle control



4. Perception

Lane Filtering

Use HSV Filtering and Canny edge detection to extract bright edge-like pixels

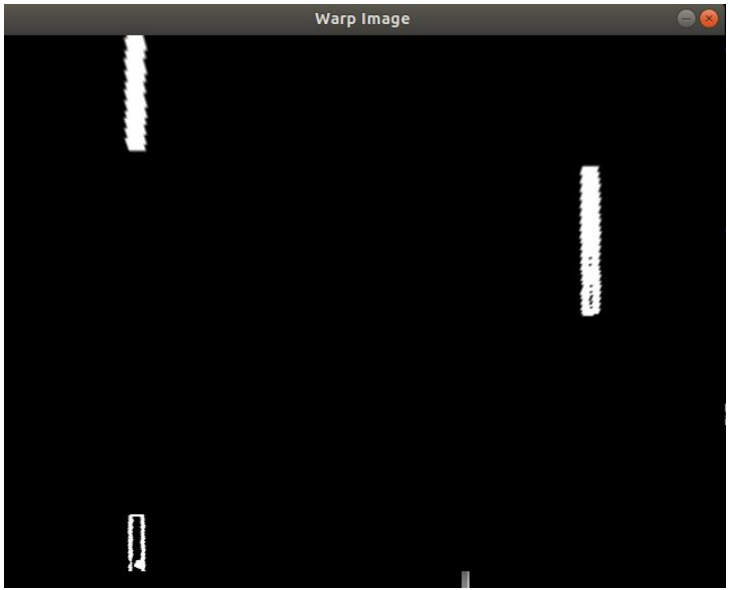


HSV Filter | Lane Feature Mask | Canny Edge Detector

4. Perception

Bird’s Eye View(BEV) Transformation

Convert the camera image into a BEV for better lane geometry extraction

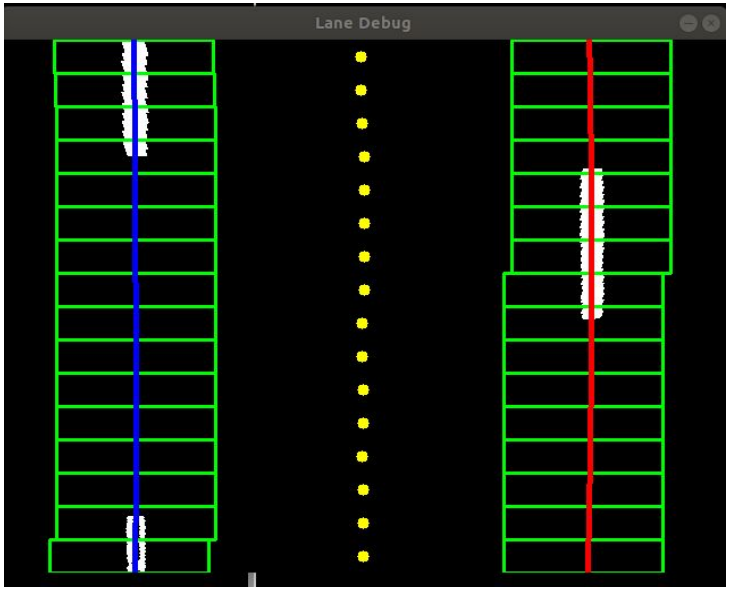


BEV Lane Map

4. Perception

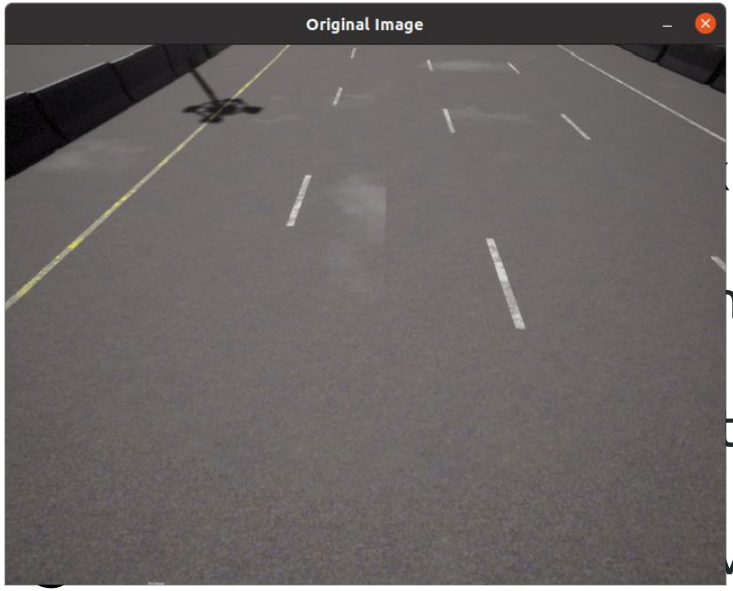
Lane Center Extraction with Sliding Window

Use Sliding window to detect lanes and apply polyfit to estimate the centerline curve



Centerline Estimation

4. Perception

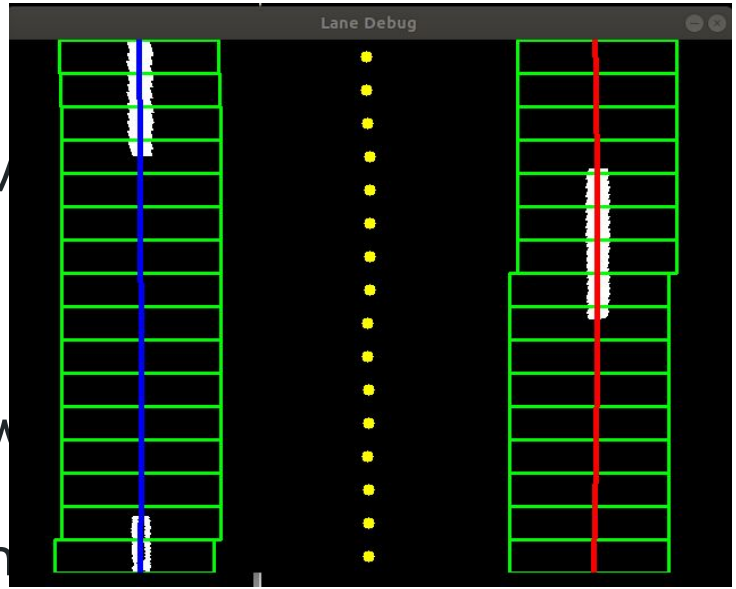


using HSV

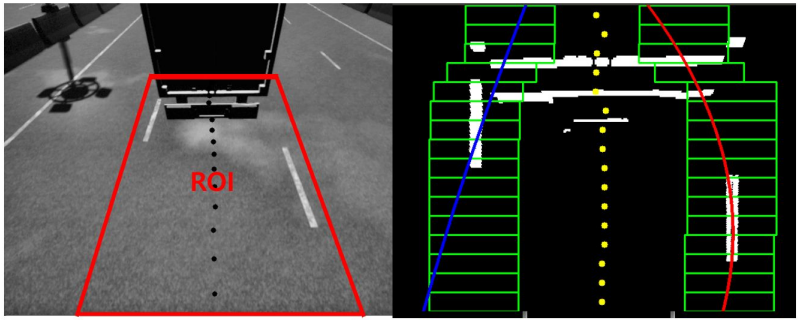
h

th sliding w

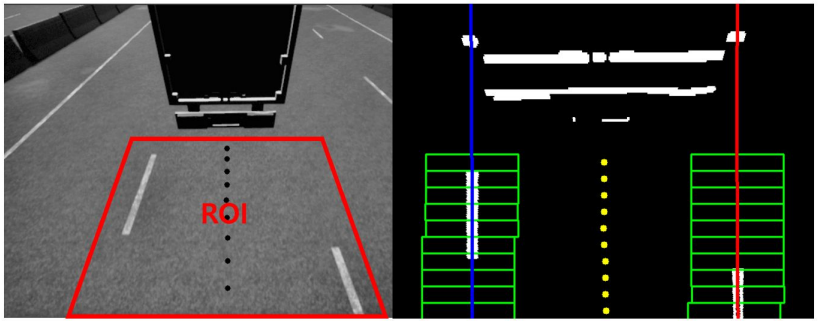
vature usin



4. Perception(Static ROI vs Dynamic ROI)



(a) 4m gap with static ROI



(b) 4m gap with dynamic ROI.

4. Perception to Control

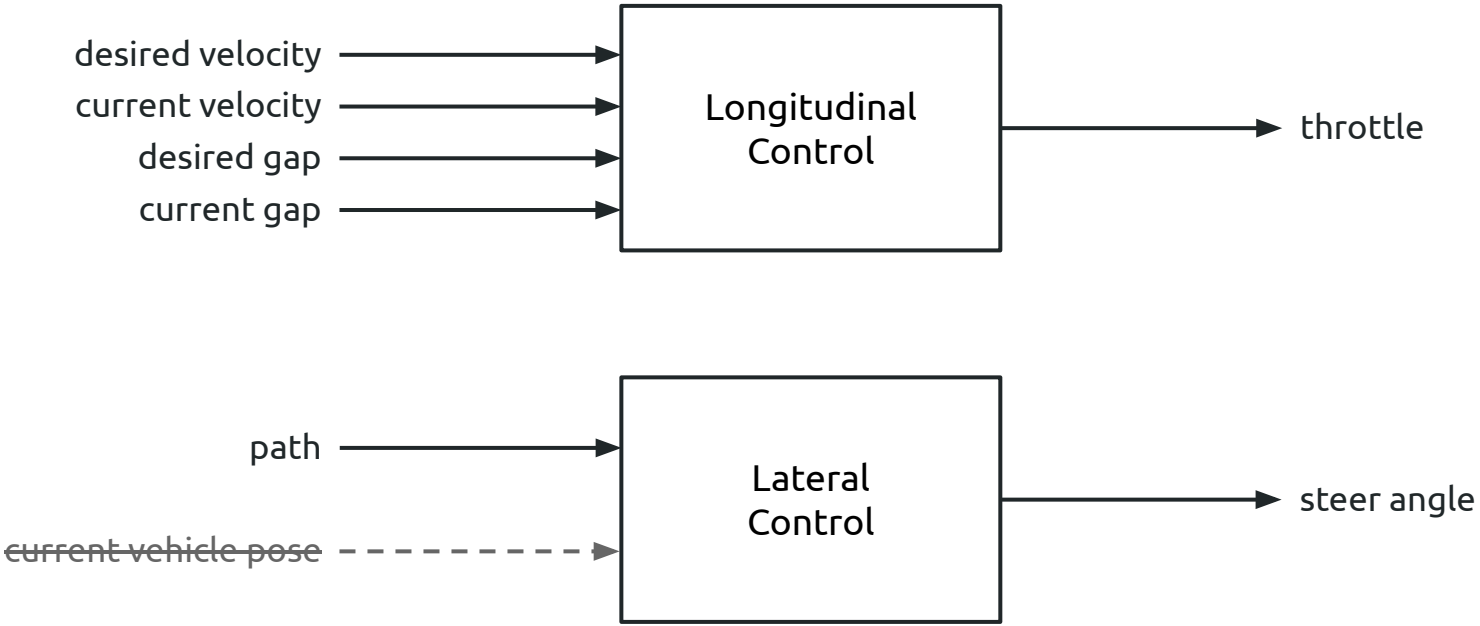


Driving Path



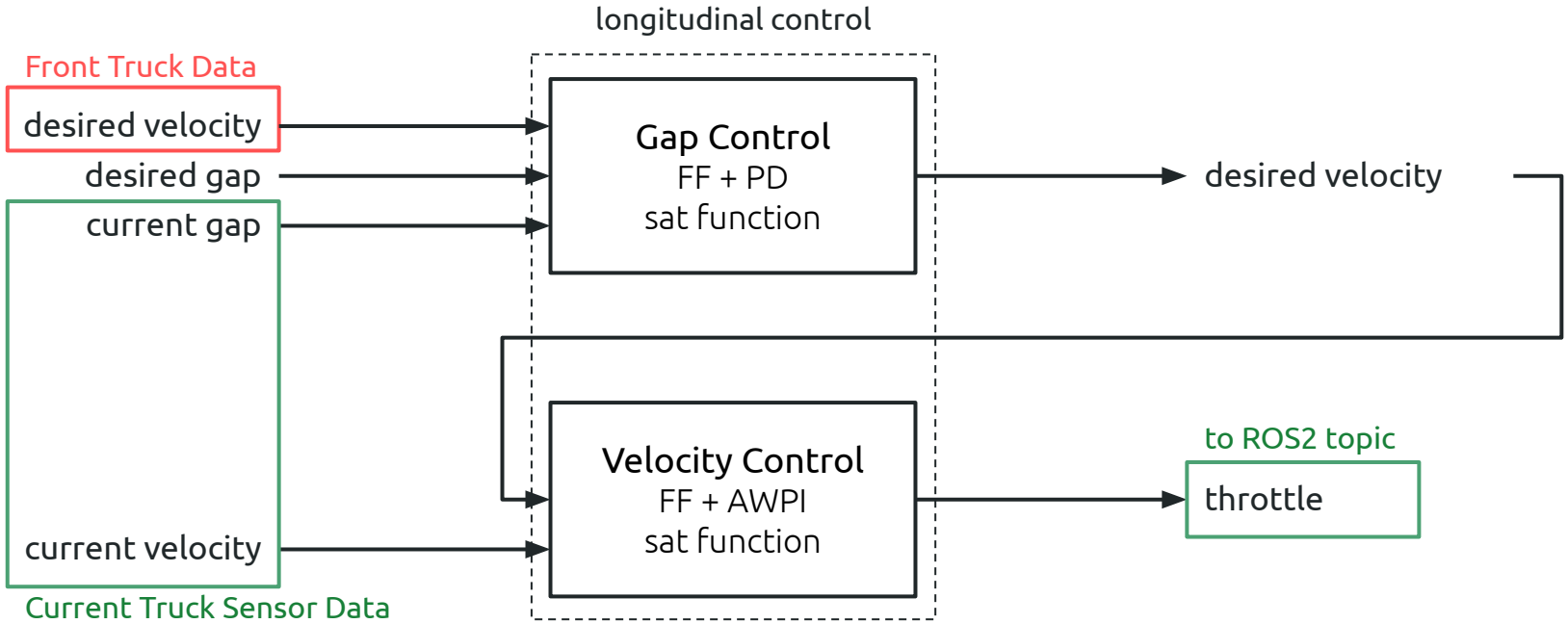
Gap Distance

5. Control


desired velocity
current velocity
desired gap
current gap
Longitudinal Control
throttle
path
current vehicle pose
Lateral Control
steer angle

5. Control (Longitudinal Control)

Overview



5. Control (Longitudinal Control)

Gap Control

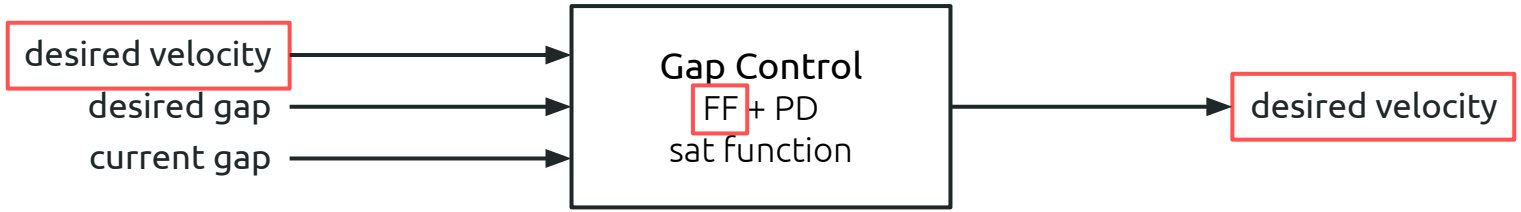
controller that adjusts the front truck’s velocity to keep a steady, safe distance



5. Control (Longitudinal Control)

Gap Control

controller that adjusts the front truck's velocity to keep a steady, safe distance



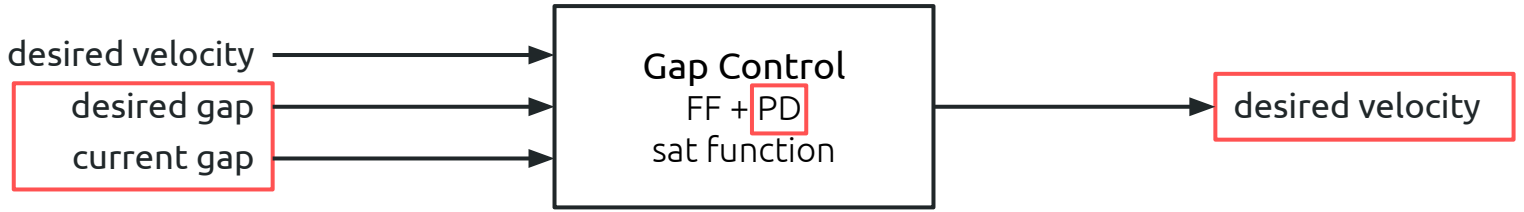
FF (Feed-Forward)

controller that proactively computes the desired output from the current input and an internal model, sending it out directly without any feedback error-correction step

5. Control (Longitudinal Control)

Gap Control

controller that adjusts the front truck’s velocity to keep a steady, safe distance



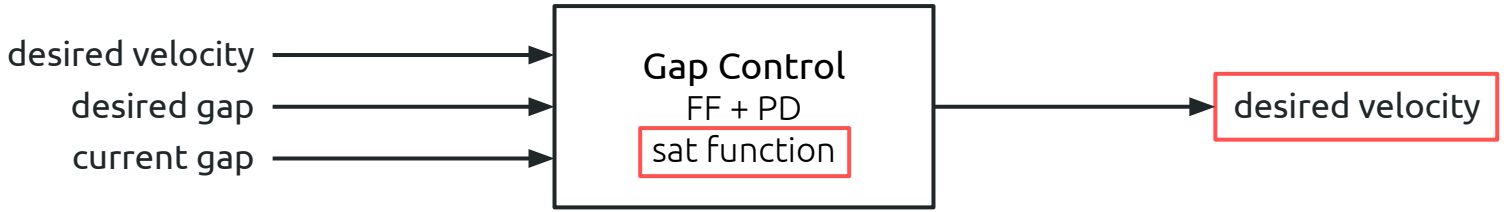
PD Control (Proportional and Derivative Control)

feedback strategy that forms the control input by weighting the current value error and its rate of change to tune the system’s response speed and stability

5. Control (Longitudinal Control)

Gap Control

controller that adjusts the front truck’s velocity to keep a steady, safe distance



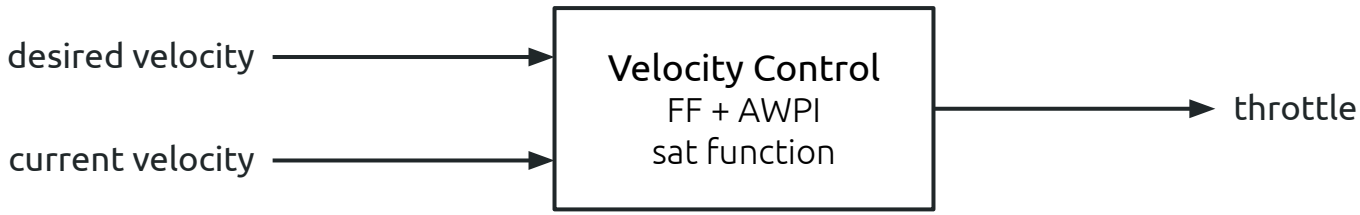
sat function (Saturation Function)

nonlinear function that clips its output to predefined upper and lower limits whenever the input exceeds those bounds, preventing excessive magnitude

5. Control (Longitudinal Control)

Velocity Control

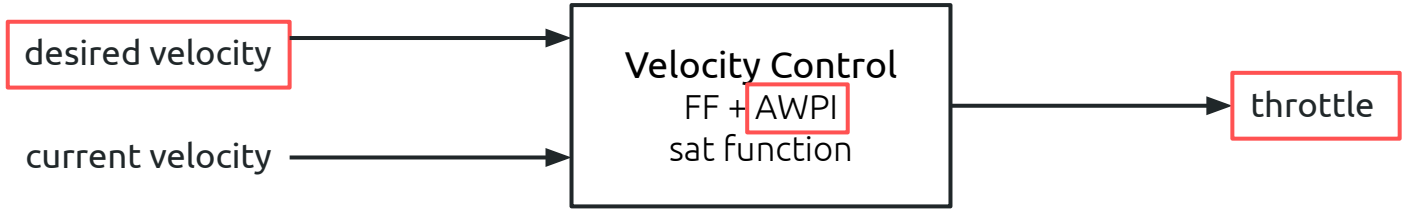
controller that regulates the truck’s power to reach and smoothly hold the desired velocity



5. Control (Longitudinal Control)

Velocity Control

controller that regulates the truck’s power to reach and smoothly hold the reference velocity



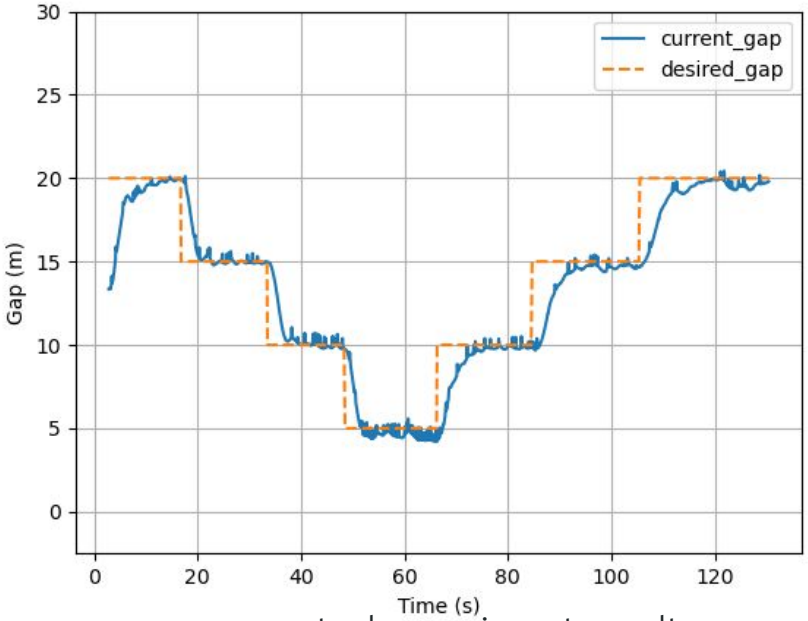
Anti-Windup PI Control

controller that augments a PI controller with anti-windup compensation to limit the integrator when the actuator saturates, thereby reducing recovery delay and transient overshoot

5. Control (Longitudinal Control)

Gap Control

controller that adjusts the front truck’s velocity to keep a steady, safe distance

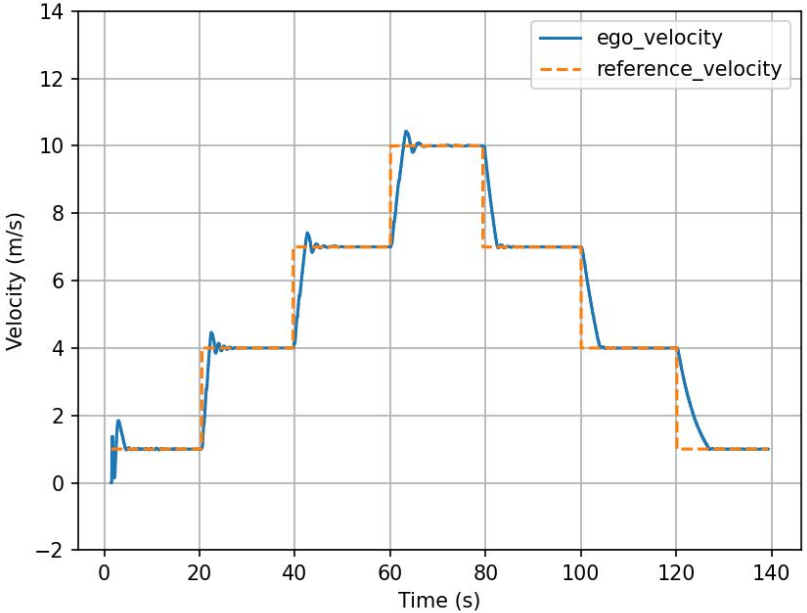


gap control experiment result

5. Control (Longitudinal Control)

Velocity Control

controller that regulates the truck’s power to reach and smoothly hold the reference velocity

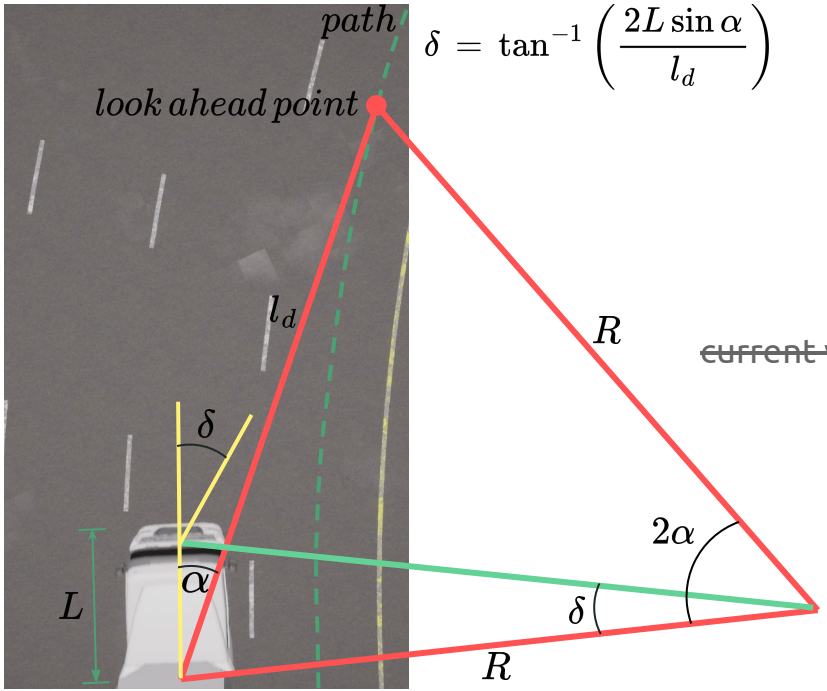


velocity control experiment result

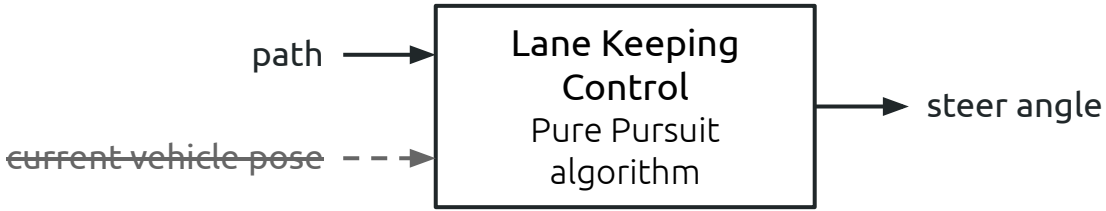
5. Control (Lateral Control)

Pure Pursuit

path tracking controller that computes steer angle based on lookahead-point



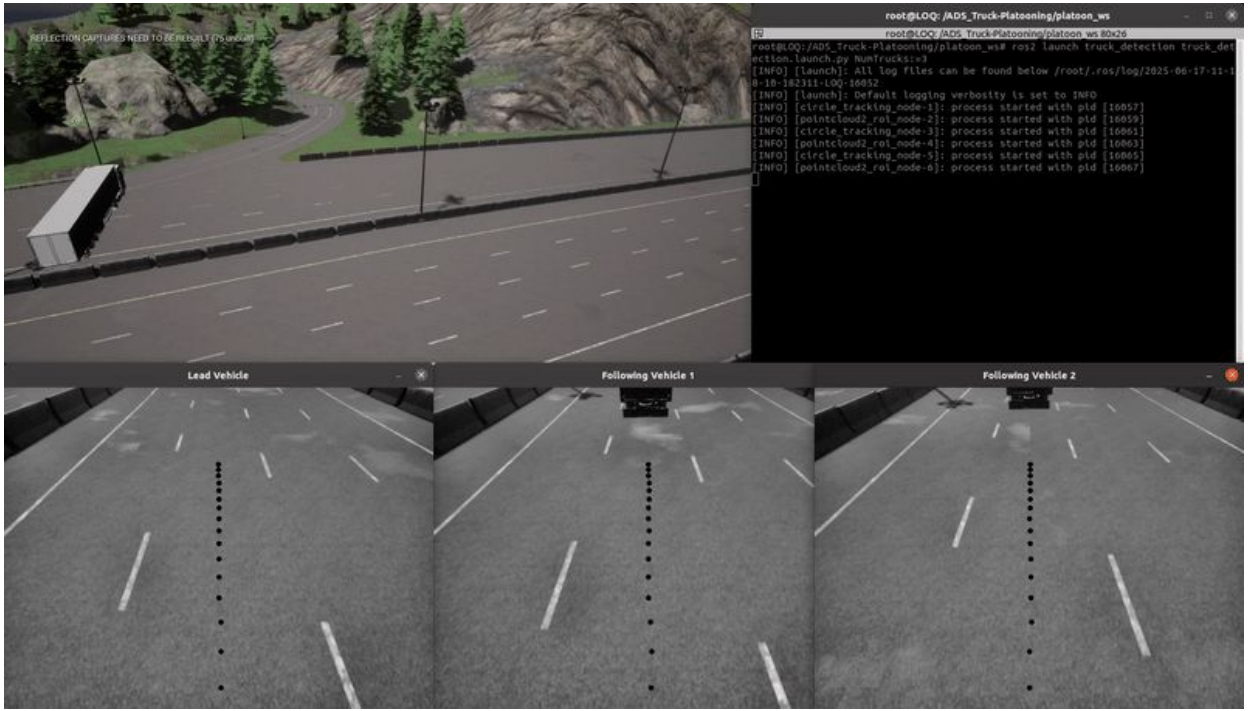
$$\delta = \tan^{-1}\left(\frac{2L\sin\alpha}{l_d}\right)$$



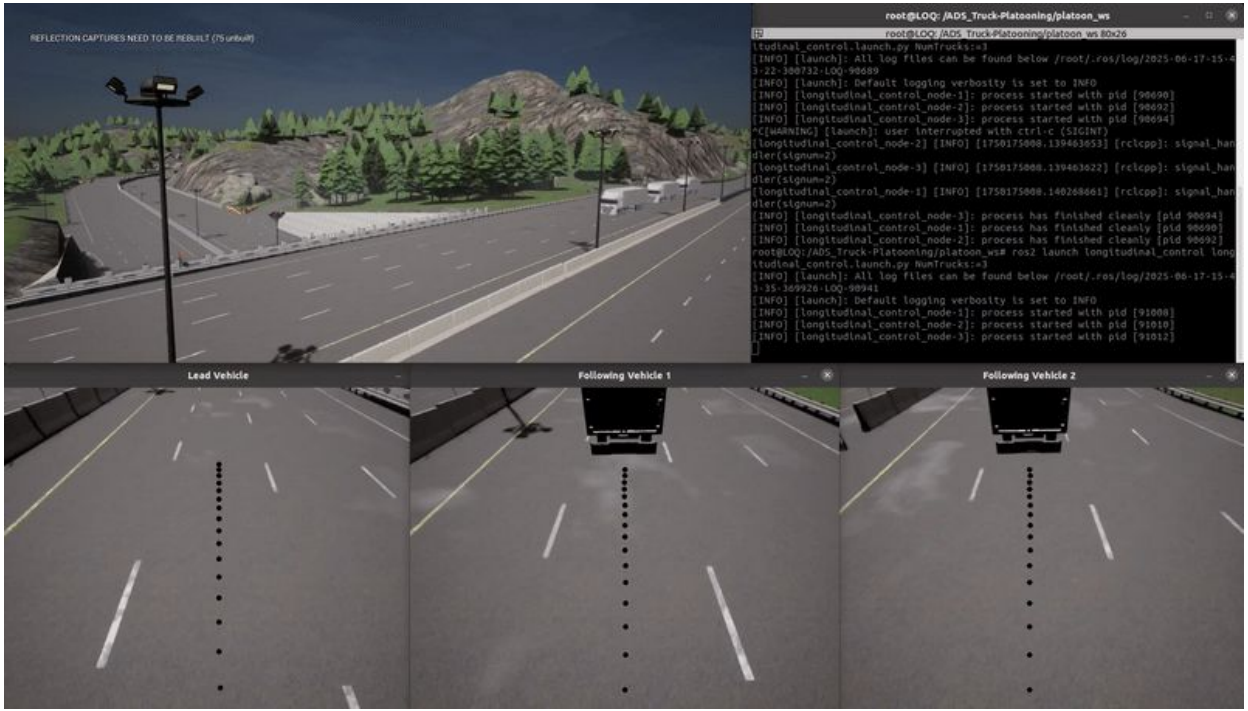
6. Scenario

- Scenario 1: Emergency Brake Activation
- Scenario 2: Camera Sensor Failure

6. Scenario 1: Emergency Brake Activation



6. Scenario 2: Camera Sensor Failure



7. Q&A



7. Q&A



7. Q&A



7. Q&A



7. Q&A



Appendix (HSV)

What is HSV?

- HSV stands for Hue, Saturation, and Value.

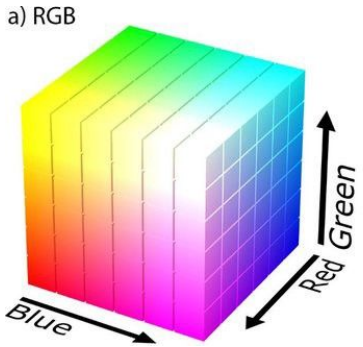
- Compared to BGR, it separates color information more clearly and is more robust to lighting changes.

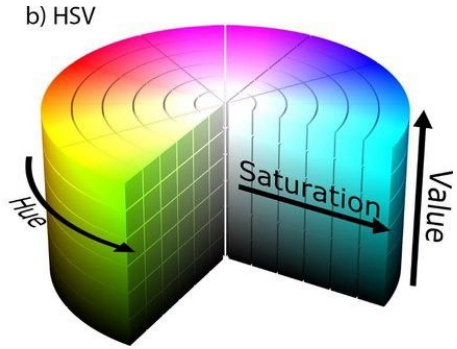
Why use HSV for Lane Detection?

- White lane markings typically have **high brightness (Value)** and **low saturation**.

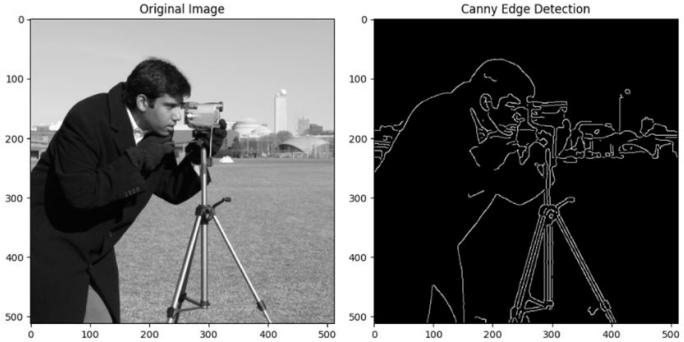
Filtering Steps

1. Convert image from **BGR to HSV** color space
2. Define a **range of HSV values** that represents white lanes
3. Keep only the **pixels within the specified range**





Appendix (Canny Edge)



1. What is Canny Edge Detection?

Canny is a **multi-stage algorithm** used to detect **edges or contours** in an image with high precision.

2. Gradient Calculation (Sobel Filter)

$$G = \sqrt{Gx^2 + Gy^2} \qquad \theta = \arctan\left(\frac{Gy}{Gx}\right)$$

Canny calculates how much the **brightness changes** at each pixel in the image.

We use the **Sobel filter**, which measures gradient (change) in two directions:

- **Gx**: Horizontal gradient (X-axis)
- **Gy**: Vertical gradient (Y-axis)

-1	-2	-1
0	0	0
1	2	1

Sobel Y

-1	0	1
-2	0	2
-1	0	1

Sobel X

Appendix (Inverse Perspective Mapping)

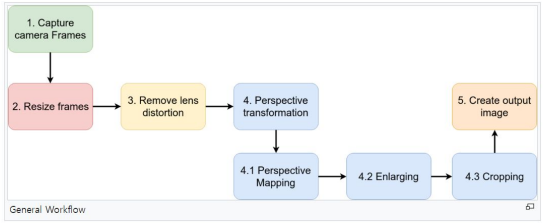
IPM

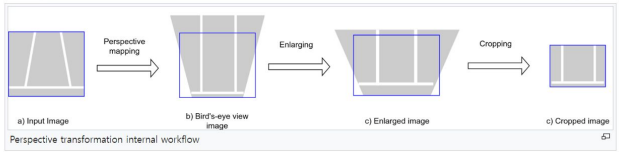
A technique that transforms a normal camera image into a top-down view projected onto a flat plane

How to do?

1. Set 4 source points and 4 destination points on the image

2. Calculate the 3×3 perspective transform matrix M and apply it to warp the image





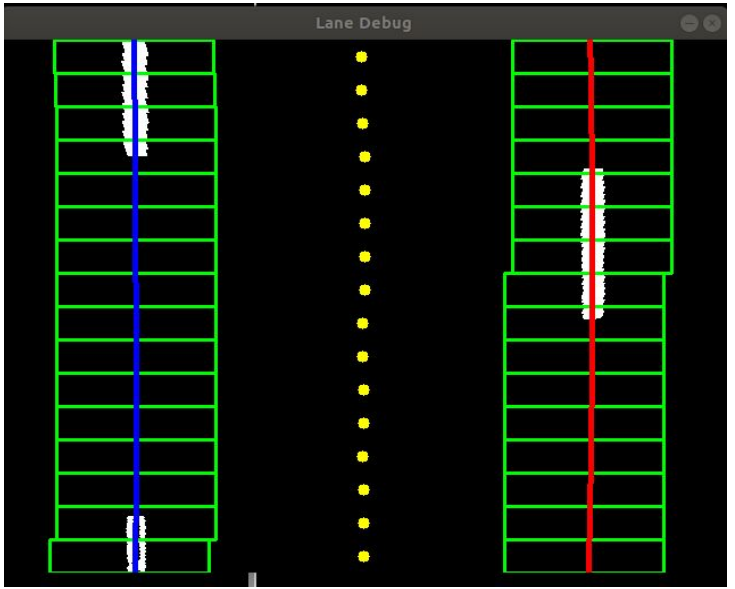
[4 fisheye camera into 360 BEV]
https://www.youtube.com/watch?v=uygu9DWiz8g&t=48s

source: RidgeRun Developer Wiki(RidgeRun's Birds Eye View project research)

Appendix(Sliding Window Method)

Move sliding windows from bottom to top of the image to follow lane positions

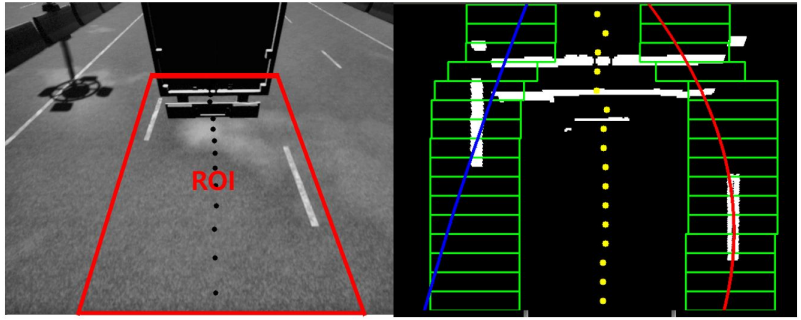
1. **Histogram analysis**: Sum pixel intensities in image columns to find the lane starting points with the highest pixel values
2. Divide the image into n horizontal windows and track the lane by shifting the window center based on detected pixels
3. Use polyfit to fit the lane points into a 2nd-degree polynomial curve



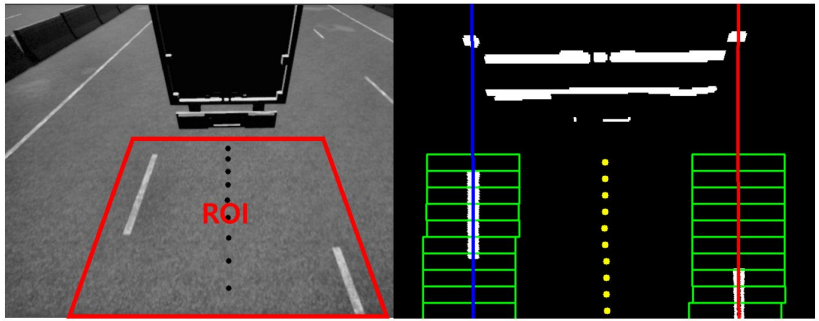
Appendix(Dynamic ROI)

Dynamic ROI Logic Flow

1. Measure the distance to the lead truck using the LiDAR sensor (in meters)
2. Convert the distance to pixels using the mapping: distance × (45 + 15 buffer)
3. Crop the BEV image from the top by that number of pixels
   → This removes the occluded region from the ROI
4. Perform lane detection only on the remaining lower area of the image



(a) 4m gap with static ROI



(b) 4m gap with dynamic ROI.

Appendix